DAT200 CA4 - Group 18

Group members: Anders Steinset Svarstad, Emilie Restrup & Thea Ariansen Gathe

In [111…

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import (
    StratifiedKFold,
    GridSearchCV,
    cross_val_predict
)
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import classification_report, ConfusionMatrixDisplay
```

Data exploration and visualization

Load and inspect data

In [112…

```python
# Load train and test
train_df = pd.read_csv('trainCA4DAT200.csv')
test_df = pd.read_csv('testCA4DAT200.csv')

# Check shapes, dtypes, .head()
print("--- Train DataFrame ---")
print(f'\nShape: {train_df.shape}')
print(f'\nData types: {train_df.dtypes}')
display(train_df.head())
print("\n--- Test DataFrame ---")
print(f'\nShape: {test_df.shape}')
print(f'\nData types: {test_df.dtypes}')
display(test_df.head())
```

```
--- Train DataFrame ---

Shape: (3500, 21)

Data types: CustomerId                int64
State                   object
AccountLength            int64
AreaCode                 int64
InternationalPlan       object
VoiceMailPlan           object
NumVoiceMailMessages     int64
DayMinutes             float64
DayCalls                 int64
DayCharge              float64
EveMinutes             float64
EveCalls                 int64
EveCharge              float64
NightMinutes           float64
NightCalls               int64
NightCharge            float64
IntlMinutes            float64
IntlCalls                int64
IntlCharge             float64
CustomerServiceCalls     int64
Churn                    int64
dtype: object
```

	CustomerId	State	AccountLength	AreaCode	InternationalPlan	VoiceMailPlan	NumVoiceMailMessages	DayMinutes	DayCalls	DayCharge	...	EveCalls	EveCharge	NightMinutes	NightCalls
0	952920	NJ	87	510	no	no	0	223.2	109	37.94	...	86	10.84	289.3	83
1	895150	RI	114	510	no	yes	28	225.8	94	38.39	...	117	16.41	232.4	100
2	636987	ND	55	415	no	no	0	180.1	77	30.62	...	98	20.02	169.4	104
3	538188	DC	100	510	yes	no	0	68.5	110	11.65	...	115	28.65	205.2	99
4	947531	AL	74	408	no	yes	46	277.7	109	47.21	...	114	22.95	165.3	82

5 rows × 21 columns

```
--- Test DataFrame ---

Shape: (1500, 20)

Data types: CustomerId                int64
State                   object
AccountLength            int64
AreaCode                 int64
InternationalPlan       object
VoiceMailPlan           object
NumVoiceMailMessages     int64
DayMinutes             float64
DayCalls                 int64
DayCharge              float64
EveMinutes             float64
EveCalls                 int64
EveCharge              float64
NightMinutes           float64
NightCalls               int64
NightCharge            float64
IntlMinutes            float64
IntlCalls                int64
IntlCharge             float64
CustomerServiceCalls     int64
dtype: object
```

	CustomerId	State	AccountLength	AreaCode	InternationalPlan	VoiceMailPlan	NumVoiceMailMessages	DayMinutes	DayCalls	DayCharge	EveMinutes	EveCalls	EveCharge	NightMinutes
0	858315	MD	125	408	no	no	0	137.1	94	23.31	209.8	83	17.83	238.4
1	583988	WY	134	510	no	no	0	296.0	93	50.32	226.4	117	19.24	246.8
2	812674	NE	103	408	no	no	0	160.8	91	27.34	155.8	82	13.24	254.3
3	611702	ID	73	408	no	no	0	67.0	76	11.39	223.0	154	18.96	236.5
4	656308	OH	162	415	no	no	0	135.2	98	22.98	242.0	107	20.57	246.9

The train and test data has all the expected columns.

Explore the raw data

In [113…

```python
# Check for missing values
print("Missing values in train dataset:")
print(train_df.isnull().sum())

# Duplicate rows
print("\nDuplicate rows in train dataset:")
print(train_df.duplicated().sum())

# Statistics summary
train_df.describe().T
```

```
Missing values in train dataset:
CustomerId              0
State                   0
AccountLength           0
AreaCode                0
InternationalPlan       0
VoiceMailPlan           0
NumVoiceMailMessages    0
DayMinutes              0
DayCalls                0
DayCharge               0
EveMinutes              0
EveCalls                0
EveCharge               0
NightMinutes            0
NightCalls              0
NightCharge             0
IntlMinutes             0
IntlCalls               0
IntlCharge              0
CustomerServiceCalls    0
Churn                   0
dtype: int64

Duplicate rows in train dataset:
0
```

Out[113…

	count	mean	std	min	25%	50%	75%	max
CustomerId	3500.0	545925.360857	261478.546087	101267.0	322381.00	536375.50	775246.7500	999391.00
AccountLength	3500.0	99.996571	39.471509	1.0	73.00	100.00	126.0000	243.00
AreaCode	3500.0	436.915429	42.169180	408.0	415.00	415.00	415.0000	510.00
NumVoiceMailMessages	3500.0	8.112000	13.728371	0.0	0.00	0.00	19.0000	52.00
DayMinutes	3500.0	180.292429	53.764503	0.0	143.70	179.70	216.5250	351.50
DayCalls	3500.0	100.016286	19.852587	0.0	87.00	100.00	113.0000	163.00
DayCharge	3500.0	30.650306	9.139923	0.0	24.43	30.55	36.8125	59.76
EveMinutes	3500.0	200.763000	50.561814	0.0	165.90	201.05	235.2250	361.80
EveCalls	3500.0	100.178000	19.800506	0.0	87.00	100.00	113.0000	170.00
EveCharge	3500.0	17.065066	4.297740	0.0	14.10	17.09	19.9925	30.75
NightMinutes	3500.0	199.984857	50.132089	0.0	166.70	199.70	233.6250	395.00
NightCalls	3500.0	99.669714	20.064302	0.0	86.00	100.00	113.0000	175.00
NightCharge	3500.0	8.999397	2.256017	0.0	7.50	8.99	10.5125	17.77
IntlMinutes	3500.0	10.297714	2.719716	0.0	8.60	10.40	12.0000	20.00
IntlCalls	3500.0	4.417143	2.456653	0.0	3.00	4.00	6.0000	19.00
IntlCharge	3500.0	2.780829	0.734263	0.0	2.32	2.81	3.2400	5.40
CustomerServiceCalls	3500.0	1.567714	1.298641	0.0	1.00	1.00	2.0000	9.00
Churn	3500.0	0.141429	0.348513	0.0	0.00	0.00	0.0000	1.00

We can see that there are no missing or duplicate values. From looking at the min, and max values for each category, we can also conclude that there are no impossible values (e.g. negative values)

In [114…

```python
# Check unique values for categorical columns

for col in ["State", "AreaCode", "InternationalPlan", "VoiceMailPlan"]:
    print(col, train_df[col].nunique())
    print(train_df[col].value_counts(dropna=False).head(10), "\n")
```

```
State 51
State
WV    121
OH     91
NY     88
AL     88
OR     81
UT     80
TX     80
MN     79
NJ     78
VA     78
Name: count, dtype: int64

AreaCode 3
AreaCode
415    1766
510     871
408     863
Name: count, dtype: int64

InternationalPlan 2
InternationalPlan
no     3168
yes     332
Name: count, dtype: int64

VoiceMailPlan 2
VoiceMailPlan
no     2530
yes     970
Name: count, dtype: int64
```

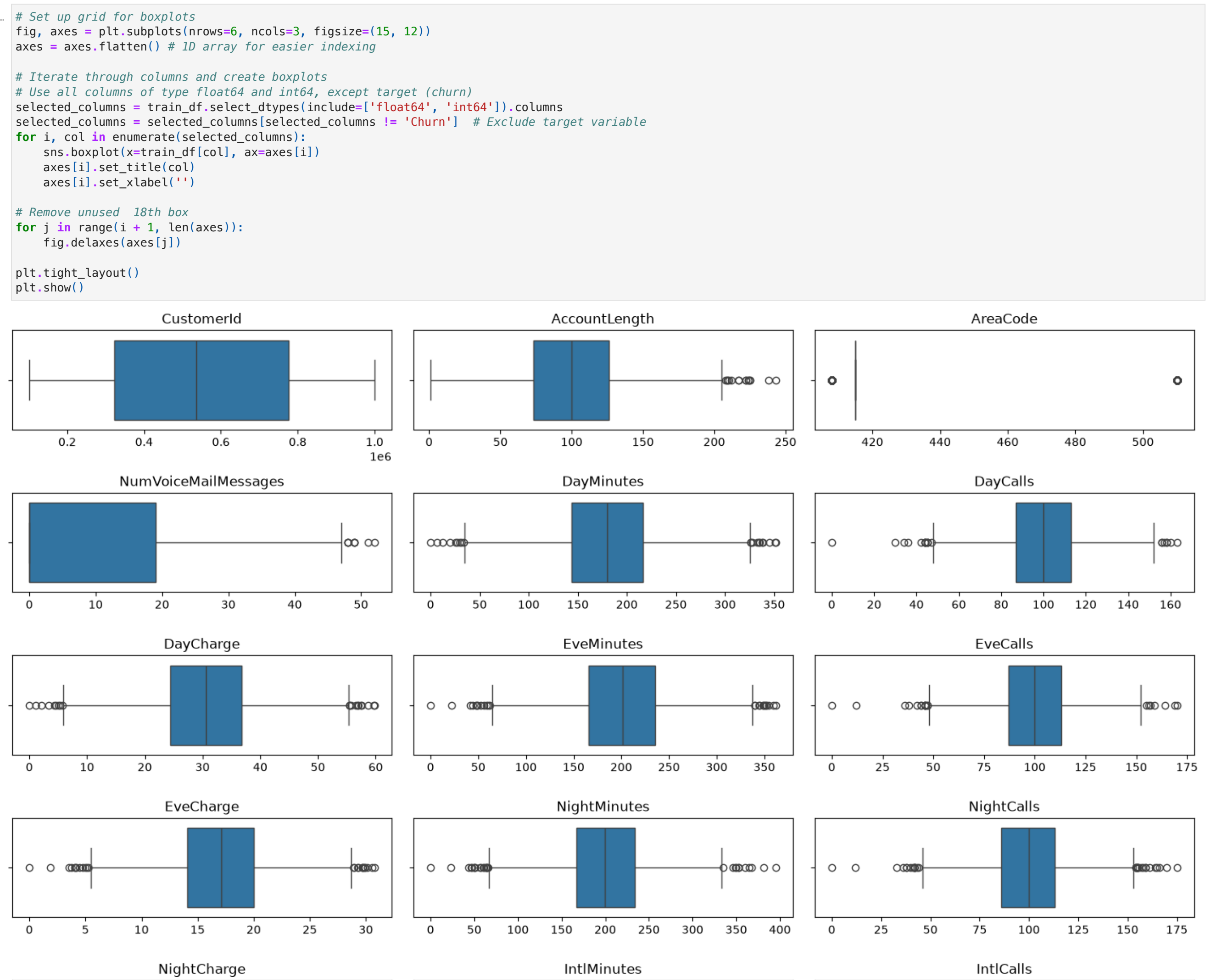
We see that we have only yes/no values for InternationalPlan and VoiceMailPlan, which means we have no impossible values here, and do not need to deal with capitalization / different spelling.

AreaCode has 3 unique values, State has 51.

Check for outliers

In [115…

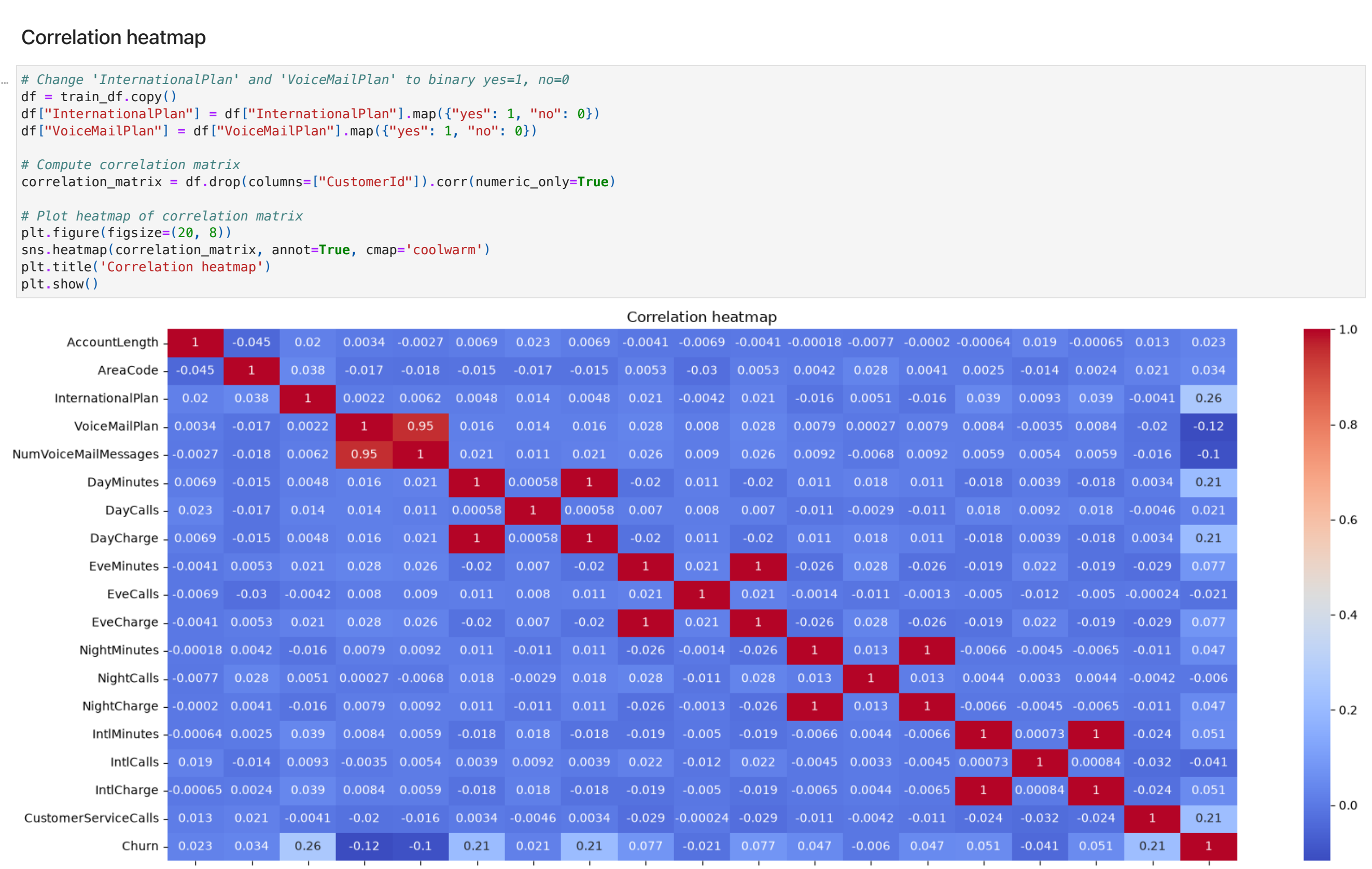
```python
# Set up grid for boxplots
fig, axes = plt.subplots(nrows=6, ncols=3, figsize=(15, 12))
axes = axes.flatten() # 1D array for easier indexing

# Iterate through columns and create boxplots
# Use all columns of type float64 and int64, except target (churn)
selected_columns = train_df.select_dtypes(include=['float64', 'int64']).columns
selected_columns = selected_columns[selected_columns != 'Churn']  # Exclude target variable
for i, col in enumerate(selected_columns):
    sns.boxplot(x=train_df[col], ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel('')

# Remove unused  18th box
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```



Correlation heatmap

In [116…

```python
# Change 'InternationalPlan' and 'VoiceMailPlan' to binary yes=1, no=0
df = train_df.copy()
df["InternationalPlan"] = df["InternationalPlan"].map({"yes": 1, "no": 0})
df["VoiceMailPlan"] = df["VoiceMailPlan"].map({"yes": 1, "no": 0})

# Compute correlation matrix
correlation_matrix = df.drop(columns=["CustomerId"]).corr(numeric_only=True)

# Plot heatmap of correlation matrix
plt.figure(figsize=(20, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation heatmap')
plt.show()
```



To determine the correlation between the features and the target we look at the churn-row and look for the highest absolute values. The correlation from highest to lowest is therefore:

1. InternationalPlan: 0.26
2. DayMinutes: 0.21
3. DayCharge: 0.21
4. CustomerServiceCalls: 0.21
5. VoiceMailPlan: -0.12
6. NumVoiceMailMessages: -0.1
7. EveMinutes: 0.077
8. EveCharge: 0.077
9. IntlMinutes: 0.051
10. IntlCharge: 0.051
11. NightMinutes: 0.047
12. NightCharge: 0.047
13. IntlCalls: -0.041
14. AreaCode: 0.034
15. AccountLength: 0.023
16. DayCalls: 0.021
17. EveCalls: -0.021
18. NightCalls: -0.006

Based on this information, we can experiment with different feature selections in training different models, and see what gives us the best results.

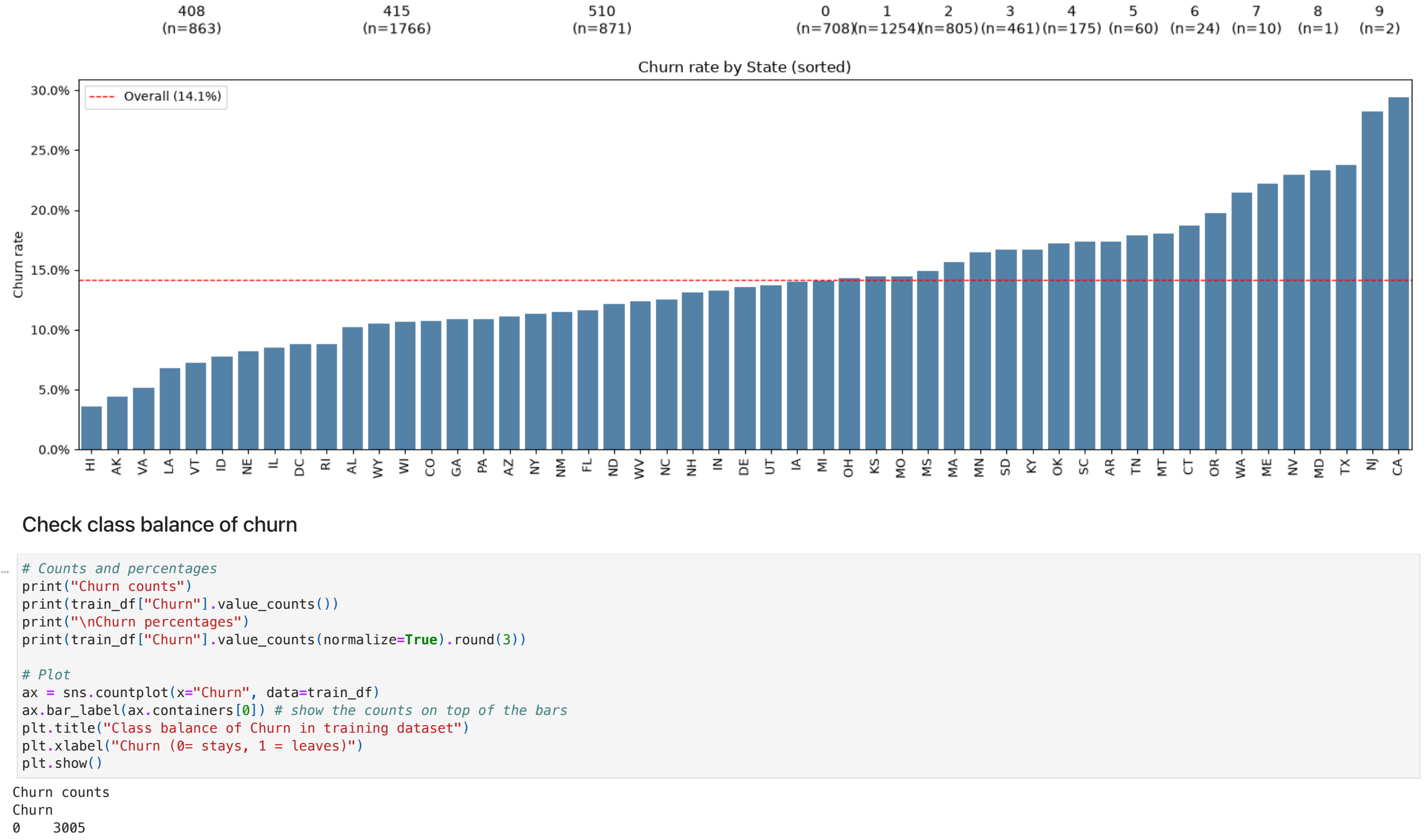
Churn rate per category

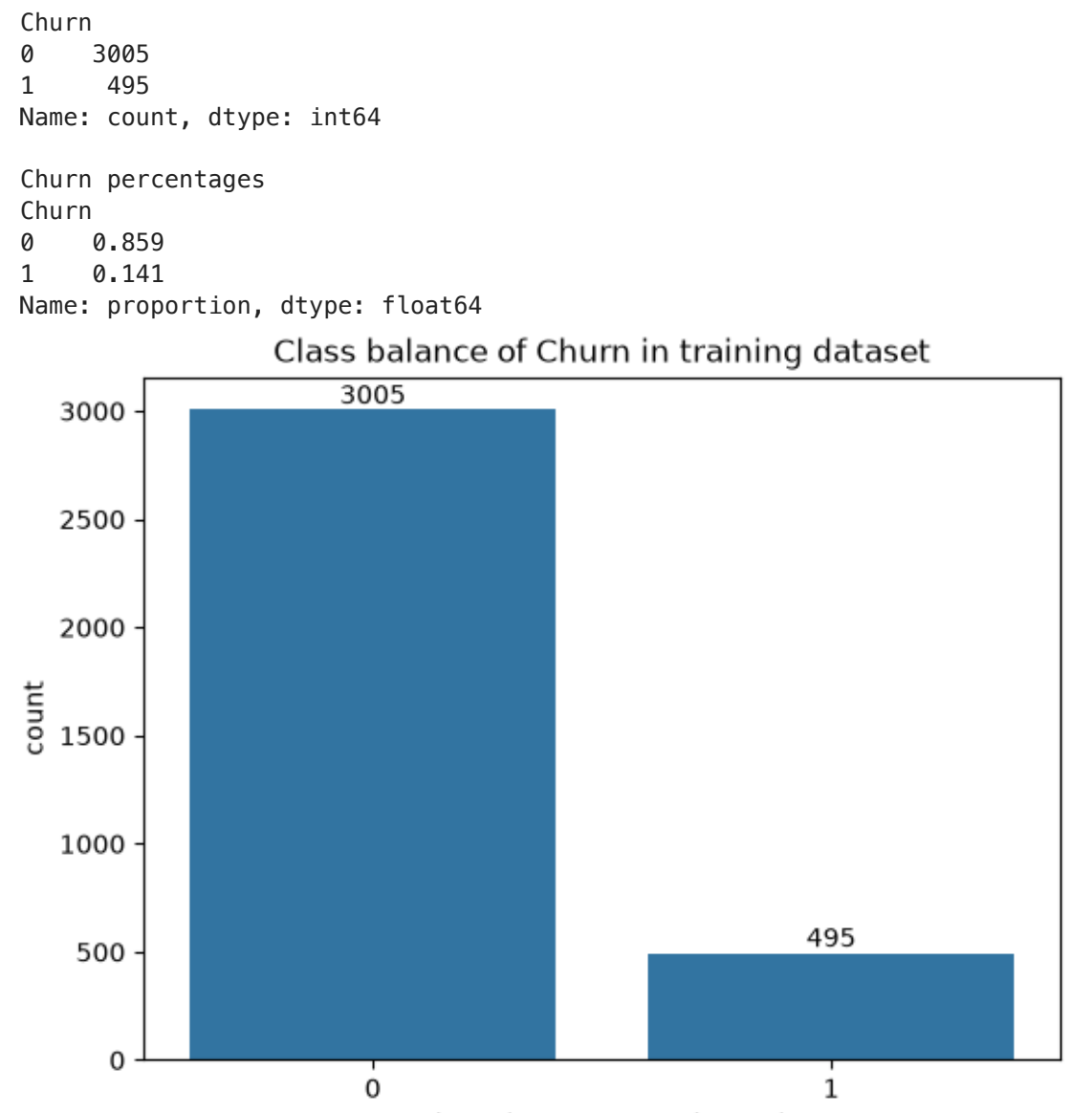
We want to look at the churn rate for some selected categories, since the correlation heatmap only shows linear relationships.

This bit of code is generated by Claude: https://claude.ai/. The comments in the code are written by us, to explain in our own words what the code does

In [117…

```python
# Calculate the overall churn rate for reference in plots
overall = train_df["Churn"].mean()

# Define a function to plot churn rate by category
def churn_rate_plot(col, ax, title=None):
    """Bar plot of churn rate per category, with group sizes (n) under the labels."""
    # Compute churn rate and group sizes
    stats = train_df.groupby(col)["Churn"].agg(rate="mean", n="size").reset_index()
    # Create bar plot
    sns.barplot(data=stats, x=col, y="rate", ax=ax, color="steelblue")
    # Format the overall churn rate line and add it to the plot
    ax.axhline(overall, color="red", linestyle="--", linewidth=1,
               label=f"Overall ({overall:.1%})")
    # Add percentage labels on top of the bars
    ax.bar_label(ax.containers[0], labels=[f"{r:.1%}" for r in stats["rate"]], fontsize=9)
    # Set x-ticks and labels with group sizes
    ax.set_xticks(range(len(stats)))
    ax.set_xticklabels([f"{c}\n(n={n})" for c, n in zip(stats[col], stats["n"])])
    # Set title and labels, format y-axis as percentage
    ax.set_title(title or f"Churn rate by {col}")
    ax.set_xlabel("")
    ax.set_ylabel("Churn rate")
    ax.yaxis.set_major_formatter(plt.matplotlib.ticker.PercentFormatter(1.0))
    # Add legend
    ax.legend()

# Plot the categorical features using the function defined above
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
churn_rate_plot("InternationalPlan", axes[0, 0])
churn_rate_plot("VoiceMailPlan", axes[0, 1])
churn_rate_plot("AreaCode", axes[1, 0])
churn_rate_plot("CustomerServiceCalls", axes[1, 1])
plt.tight_layout()
plt.show()

# Make a bigger plot for State, sorted by churn rate
# Compute the churn rate and group sizes for each state, then sort by churn rate
state_stats = (train_df.groupby("State")["Churn"]
               .agg(rate="mean", n="size")
               .sort_values("rate")
               .reset_index())
# Create a bar plot
fig, ax = plt.subplots(figsize=(16, 5))
sns.barplot(data=state_stats, x="State", y="rate", ax=ax, color="steelblue")
# Add overall churn rate line
ax.axhline(overall, color="red", linestyle="--", linewidth=1,
           label=f"Overall ({overall:.1%})")
# Add title, labels, and format y-axis as percentage, rotate x--ticks and add legend
ax.set_title("Churn rate by State (sorted)")
ax.set_ylabel("Churn rate")
ax.set_xlabel("")
ax.yaxis.set_major_formatter(plt.matplotlib.ticker.PercentFormatter(1.0))
plt.xticks(rotation=90)
ax.legend()
plt.tight_layout()
plt.show()
```



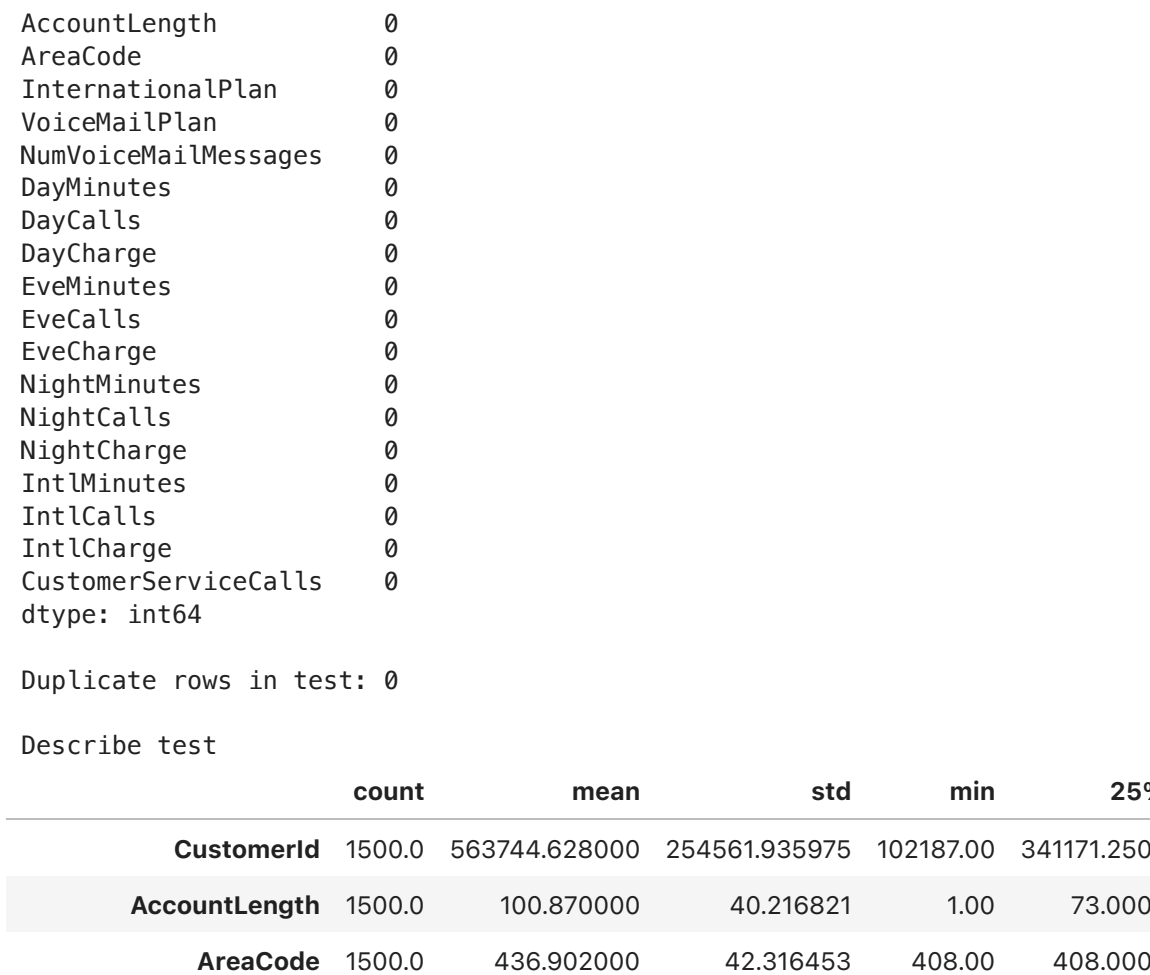


Check class balance of churn

In [118…

```python
# Counts and percentages
print("Churn counts")
print(train_df["Churn"].value_counts())
print("\nChurn percentages")
print(train_df["Churn"].value_counts(normalize=True).round(3))

# Plot
ax = sns.countplot(x="Churn", data=train_df)
ax.bar_label(ax.containers[0]) # show the counts on top of the bars
plt.title("Class balance of Churn in training dataset")
plt.xlabel("Churn (0= stays, 1 = leaves)")
plt.show()
```

```
Churn counts
Churn
0    3005
1     495
Name: count, dtype: int64

Churn percentages
Churn
0    0.859
1    0.141
Name: proportion, dtype: float64
```



The churn rate is 14.1%, so the data is imbalanced. This is something we have to take into account when making the models.

Check test set

In [119…

```python
# Missing, duplicate and impossible values
print("Missing values in test:\n", test_df.isnull().sum())
print("\nDuplicate rows in test:", test_df.duplicated().sum())
print("\nDescribe test")
display(test_df.describe().T)
```

```
Missing values in test:
 CustomerId              0
State                   0
AccountLength           0
AreaCode                0
InternationalPlan       0
VoiceMailPlan           0
NumVoiceMailMessages    0
DayMinutes              0
DayCalls                0
DayCharge               0
EveMinutes              0
EveCalls                0
EveCharge               0
NightMinutes            0
NightCalls              0
NightCharge             0
IntlMinutes             0
IntlCalls               0
IntlCharge              0
CustomerServiceCalls    0
dtype: int64

Duplicate rows in test: 0

Describe test
```

	count	mean	std	min	25%	50%	75%	max
CustomerId	1500.0	563744.628000	254561.935975	102187.00	341171.2500	583238.000	777220.5000	999302.00
AccountLength	1500.0	100.870000	40.216821	1.00	73.0000	100.500	128.0000	233.00
AreaCode	1500.0	436.902000	42.316453	408.00	408.0000	415.000	438.7500	510.00
NumVoiceMailMessages	1500.0	6.922667	13.078636	0.00	0.0000	0.000	0.0000	50.00
DayMinutes	1500.0	180.280667	54.215261	0.00	143.6750	181.500	215.9000	346.80
DayCalls	1500.0	100.060000	19.787777	0.00	87.0000	100.000	114.0000	165.00
DayCharge	1500.0	30.648180	9.216592	0.00	24.4250	30.860	36.7000	58.96
EveMinutes	1500.0	200.341533	50.542416	31.20	167.1750	200.950	232.1250	363.70
EveCalls	1500.0	100.221333	19.893588	37.00	87.0000	100.000	114.0000	168.00
EveCharge	1500.0	17.029253	4.296079	2.65	14.2075	17.085	19.7325	30.91
NightMinutes	1500.0	201.340733	51.443637	45.00	167.5250	201.650	236.3000	381.90
NightCalls	1500.0	100.501333	19.704403	46.00	88.0000	100.000	114.0000	168.00
NightCharge	1500.0	9.060513	2.314835	2.03	7.5375	9.075	10.6300	17.19
IntlMinutes	1500.0	10.177933	2.855450	0.00	8.3750	10.300	12.1000	18.90
IntlCalls	1500.0	4.477333	2.457406	0.00	3.0000	4.000	6.0000	20.00
IntlCharge	1500.0	2.748720	0.770909	0.00	2.2625	2.780	3.2700	5.10
CustomerServiceCalls	1500.0	1.576667	1.324623	0.00	1.0000	1.000	2.0000	8.00

No missing, duplicate or impossible values in test set.

In [120…

```python
# Check for new categories that dont appear in the training set
for col in ["State", "AreaCode", "InternationalPlan", "VoiceMailPlan"]:
    print(col, "-> only in test:", set(test_df[col]) - set(train_df[col]))
```

```
State -> only in test: set()
AreaCode -> only in test: set()
InternationalPlan -> only in test: set()
VoiceMailPlan -> only in test: set()
```

No new categories

In [121…

```python
# Check that CustomerId is unique and dont overlap between the train and test set

print("\nDuplicate IDs in train:", train_df["CustomerId"].duplicated().sum())
print("IDs in both train and test:", len(set(train_df["CustomerId"]) & set(test_df["CustomerId"])))
```

```
Duplicate IDs in train: 0
IDs in both train and test: 0
```

No duplicate CustomerIds between the train and test sets

Data cleaning

As we saw, there were no missing, duplicate or impossible values in either the train set or test set. We will keep outliers since they are plausible values and can contribute meaningful information to the model. For example, we see in the plot for churn rates by CustomerServiceCall that the rate increases significantly for higher numbers of calls.

For the cleaning we will do the following:

- Change InternationalPlan and VoiceMailPlan to binary (yes=1, no=0)
- AreaCode to string, so its treated as a category
- Drop CustomerId from the features, but keep the testIDs for the Kaggle submission

We create a function, so it can easily be reused when training our models.

In [122…

```python
def clean(df):
    df = df.copy()

    # Convert InternationalPlan and VoiceMailPlan to binary
    for col in ["InternationalPlan", "VoiceMailPlan"]:
        df[col] = df[col].map({"yes": 1, "no": 0})

    # Convert AreaCode to string
    df["AreaCode"] = df["AreaCode"].astype(str)

    # Drop CustomerId, since it is an identifier and not a feature
    df = df.drop(columns=["CustomerId"], errors="ignore")

    # Double check that there are no missing values
    assert df.isnull().sum().sum() == 0

    return df
```

Create Cross-Validation setup

Use StratifiedKFold from Scikit-learn to create cross-validation setup to be used when training models.

- 5 folds of 700 customers each. Since there are only about 490 churners, this is a good amount of folds to ensure there are enough churners in each validation fold
- Shuffle the rows before splitting, so folds don't depend on the order of the file
- use random_state=42, so the shuffle is reproducible and all group members get identical folds
- Stratified means that every fold keeps the same churn percentage as the full training set, 14.1%.

In [123…

```python
RANDOM_STATE = 42
SCORING = "f1_macro" # primary evaluation metric

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
```

Modelling

We make two required pipelines: one with regularization (logistic regression) and one with a kernel and dimensionality reduction (PCA + SVM). We also try gradient boosting to see if we can get a better model. All models are tuned with GridSearchCV using the same `cv` and `SCORING` as defined above, so the comparison is fair.

The code for the modelling is generated with help from Claude: https://claude.ai/.

Prepare data

In [124…

```python
# Clean train and test with the function from the data cleaning
X = clean(train_df.drop(columns=["Churn"]))
y = train_df["Churn"]
X_test = clean(test_df)
test_ids = test_df["CustomerId"]  # keep for the Kaggle submission
```

Feature engineering

The Charge columns are just the Minutes multiplied by a fixed price (correlation 1.0), so they give no new information. Instead we make some new features:

- Total minutes, total calls and total charge over the whole day, since customers with a high total bill may be more likely to leave
- A flag for 4 or more customer service calls, since we saw that the churn rate increases a lot from 4 calls

In [125…

```python
def add_features(df):
    df = df.copy()
    periods = ["Day", "Eve", "Night", "Intl"]

    # Totals over all periods
    df["TotalMinutes"] = df[[p + "Minutes" for p in periods]].sum(axis=1)
    df["TotalCalls"] = df[[p + "Calls" for p in periods]].sum(axis=1)
    df["TotalCharge"] = df[[p + "Charge" for p in periods]].sum(axis=1)

    # Flag for many customer service calls
    df["ManyServiceCalls"] = (df["CustomerServiceCalls"] >= 4).astype(int)

    # Drop the charge columns for each period
    df = df.drop(columns=[p + "Charge" for p in periods])
    return df

X = add_features(X)
X_test = add_features(X_test)
X.head()
```

Out[125…

	State	AccountLength	AreaCode	InternationalPlan	VoiceMailPlan	NumVoiceMailMessages	DayMinutes	DayCalls	EveMinutes	EveCalls	NightMinutes	NightCalls	IntlMinutes	IntlCalls
0	NJ	87	510	0	0	0	223.2	109	127.5	86	289.3	83	14.5	4
1	RI	114	510	0	1	28	225.8	94	193.0	117	232.4	100	8.4	9
2	ND	55	415	0	0	0	180.1	77	235.5	98	169.4	104	11.1	2
3	DC	100	510	1	0	0	68.5	110	337.1	115	205.2	99	12.1	9
4	AL	74	408	0	1	46	277.7	109	270.0	114	165.3	82	10.4	2

Preprocessing

- State and AreaCode are categorical, so they are one-hot encoded
- The numerical features are standardized, which is important for logistic regression and SVM

The preprocessing is put inside each pipeline, so it is only fitted on the training folds in the cross-validation (no data leakage).

In [126…

```python
cat_cols = ["State", "AreaCode"]
num_cols = [col for col in X.columns if col not in cat_cols]

preprocessor = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_cols),
    ("num", StandardScaler(), num_cols),
])
```

Pipeline 1: Logistic regression (regularization)

`C` is the inverse of the regularization strength (small C = strong regularization). `l1_ratio=0` gives L2 regularization and `l1_ratio=1` gives L1 regularization, which can set coefficients to zero. We use `class_weight="balanced"` because the classes are imbalanced.

In [127…

```
pipe_lr = Pipeline([
    ("preprocess", preprocessor),
    ("clf", LogisticRegression(solver="saga", class_weight="balanced", max_iter=5000))
])

param_grid_lr = {
    "clf__C": [0.001, 0.003, 0.01, 0.1, 1, 10],
    "clf__l1_ratio": [0, 1],  # 0 = L2, 1 = L1
}

grid_lr = GridSearchCV(pipe_lr, param_grid_lr, scoring=SCORING, cv=cv, n_jobs=-1)
grid_lr.fit(X, y)

print("Best score:", round(grid_lr.best_score_, 4))
print("Best params:", grid_lr.best_params_)
```

```
Best score: 0.7816
Best params: {'clf__C': 0.003, 'clf__l1_ratio': 1}
```

The best logistic regression gets a macro F1 of 0.78. It uses strong L1 regularization (C=0.003), which sets many of the coefficients to zero. The score is quite low, which tells us that a linear model is not able to capture the relationship between the features and churn very well.

Pipeline 2: PCA + SVM (kernel and dimensionality reduction)

After one-hot encoding there are about 70 features, most of them from State. PCA reduces the number of features, and the SVM with RBF kernel can find a non-linear decision boundary. We tune the number of PCA components together with `C` and `gamma`.

In [128…

```
pipe_svm = Pipeline([
    ("preprocess", preprocessor),
    ("pca", PCA(random_state=RANDOM_STATE)),
    ("clf", SVC(kernel="rbf"))
])

param_grid_svm = {
    "pca__n_components": [10, 20, 40],
    "clf__C": [1, 10, 100],
    "clf__gamma": ["scale", 0.01, 0.03],
    "clf__class_weight": [None, "balanced"],
}

grid_svm = GridSearchCV(pipe_svm, param_grid_svm, scoring=SCORING, cv=cv, n_jobs=-1)
grid_svm.fit(X, y)

print("Best score:", round(grid_svm.best_score_, 4))
print("Best params:", grid_svm.best_params_)
```

```
Best score: 0.8662
Best params: {'clf__C': 10, 'clf__class_weight': None, 'clf__gamma': 0.03, 'pca__n_components': 20}
```

PCA + SVM gets a macro F1 of 0.87, which is a lot better than logistic regression. The best model uses 20 PCA components. Since the SVM with RBF kernel can make a non-linear decision boundary, this confirms that the relationship is non-linear.

Pipeline 3: Gradient boosting (better model)

Since the relationship between the features and churn seems to be non-linear, we also try a tree-based model. Gradient boosting builds many small decision trees, where each new tree corrects the errors of the previous ones.

In [129…

```
pipe_gb = Pipeline([
    ("preprocess", preprocessor),
    ("clf", HistGradientBoostingClassifier(random_state=RANDOM_STATE))
])

param_grid_gb = {
    "clf__learning_rate": [0.05, 0.1],
    "clf__max_iter": [200, 400],
    "clf__min_samples_leaf": [20, 50],
    "clf__class_weight": [None, "balanced"],
}

grid_gb = GridSearchCV(pipe_gb, param_grid_gb, scoring=SCORING, cv=cv, n_jobs=-1)
grid_gb.fit(X, y)

print("Best score:", round(grid_gb.best_score_, 4))
print("Best params:", grid_gb.best_params_)
```

```
Best score: 0.9529
Best params: {'clf__class_weight': 'balanced', 'clf__learning_rate': 0.05, 'clf__max_iter': 400, 'clf__min_samples_leaf': 20}
```

Comparing

In [130…

```
results = pd.DataFrame({
    "Model": ["Logistic regression", "PCA + SVM", "Gradient boosting"],
    "Best CV macro F1": [grid_lr.best_score_, grid_svm.best_score_, grid_gb.best_score_]
})
results = results.sort_values("Best CV macro F1", ascending=False)
results
```

Out[130…

	Model	Best CV macro F1
2	Gradient boosting	0.952918
1	PCA + SVM	0.866223
0	Logistic regression	0.781612

Gradient boosting is clearly the best model with a macro F1 of 0.95, and is chosen as our final model. Both required pipelines perform worse, but PCA + SVM is better than logistic regression.

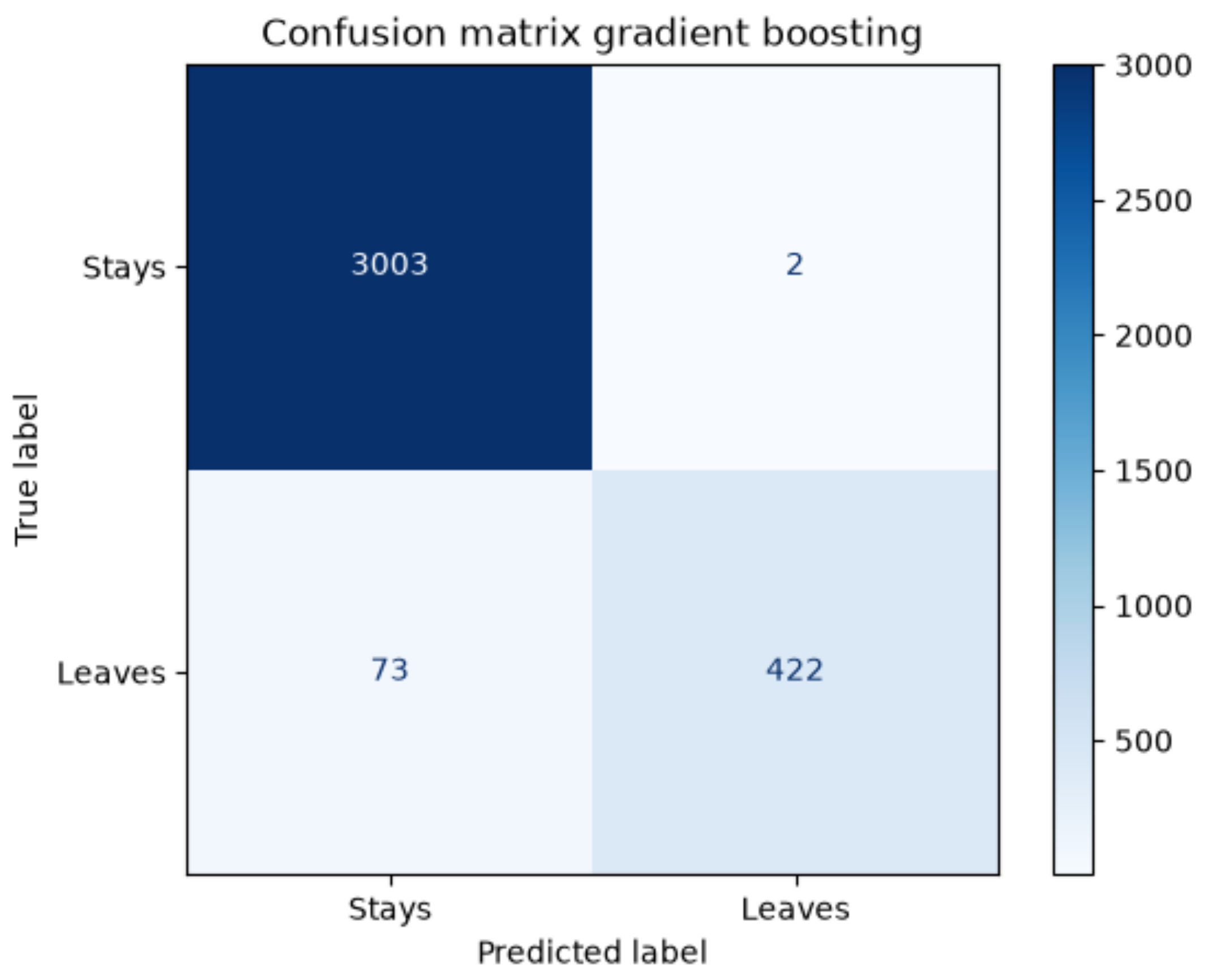
Final evaluation

To evaluate the best model we use `cross_val_predict`, so each customer is predicted by a model that has not seen that customer during training.

In [131…

```
best_model = grid_gb.best_estimator_

# Predictions from cross-validation
y_pred = cross_val_predict(best_model, X, y, cv=cv)

print(classification_report(y, y_pred, target_names=["Stays (0)", "Leaves (1)"]))
ConfusionMatrixDisplay.from_predictions(y, y_pred, display_labels=["Stays", "Leaves"], cmap="Blues")
plt.title("Confusion matrix gradient boosting")
plt.show()
```

```
              precision    recall  f1-score   support

   Stays (0)       0.98      1.00      0.99      3005
  Leaves (1)       1.00      0.85      0.92       495

    accuracy                           0.98      3500
   macro avg       0.99      0.93      0.95      3500
weighted avg       0.98      0.98      0.98      3500
```



The model is very good at predicting the customers who stay. When it predicts that a customer leaves, it is almost always correct (precision 1.00 for churn). However, it misses about 15 % of the customers who actually leave (recall 0.85).

Kaggle submission

In [132…

```
# Train the best model on the whole training set and predict the test set
best_model.fit(X, y)
test_pred = best_model.predict(X_test)

submission = pd.DataFrame({"CustomerId": test_ids, "Churn": test_pred})
submission.to_csv("submission.csv", index=False)

print(submission["Churn"].value_counts())
submission.head()
```

```
Churn
0    1316
1     184
Name: count, dtype: int64
```

Out[132…

	CustomerId	Churn
0	858315	0
1	583988	1
2	812674	0
3	611702	0
4	656308	0

Conclusion

We made two required pipelines, logistic regression with regularization and PCA + SVM with an RBF kernel, and in addition a gradient boosting model. All models were tuned with GridSearchCV and evaluated with the same stratified 5-fold cross-validation and macro F1.

- Logistic regression: 0.78
- PCA + SVM: 0.87
- Gradient boosting: 0.95

Gradient boosting performed best and was used for the Kaggle submission, which beats the BEAT ME benchmark. The new features are based on what we found in the data exploration: the Charge columns give no extra information compared to the Minutes columns, and the churn rate increases a lot for customers with 4 or more customer service calls. The model still misses some of the customers who leave, which is common when the classes are imbalanced.